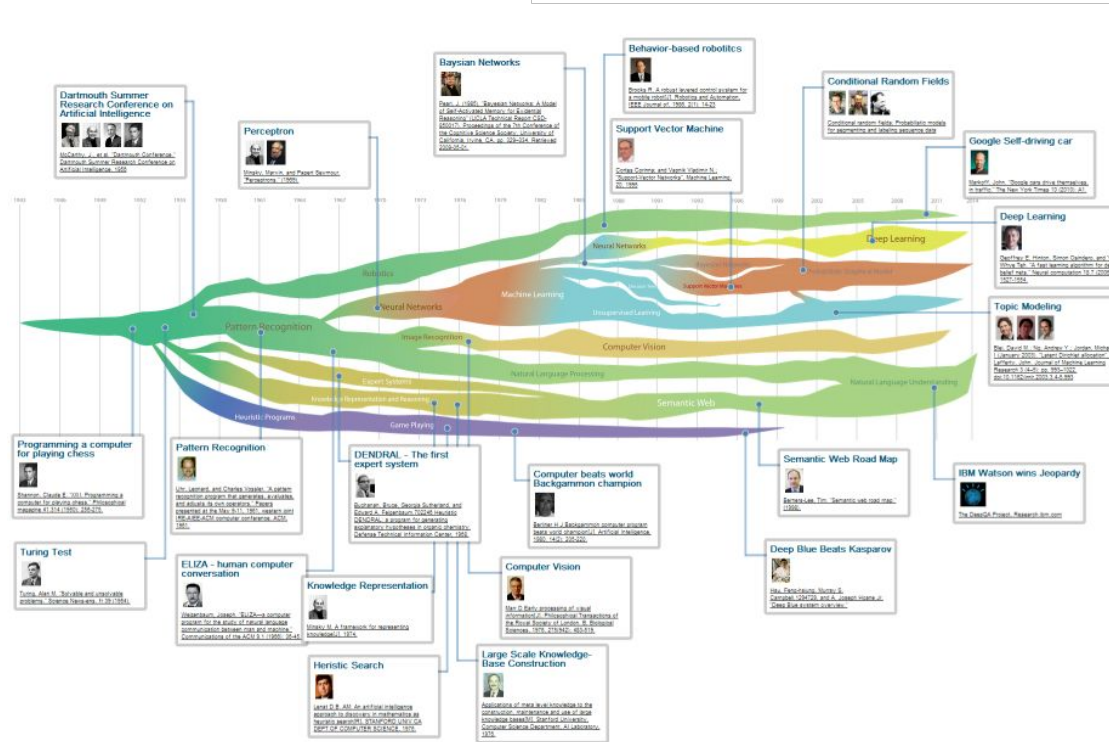


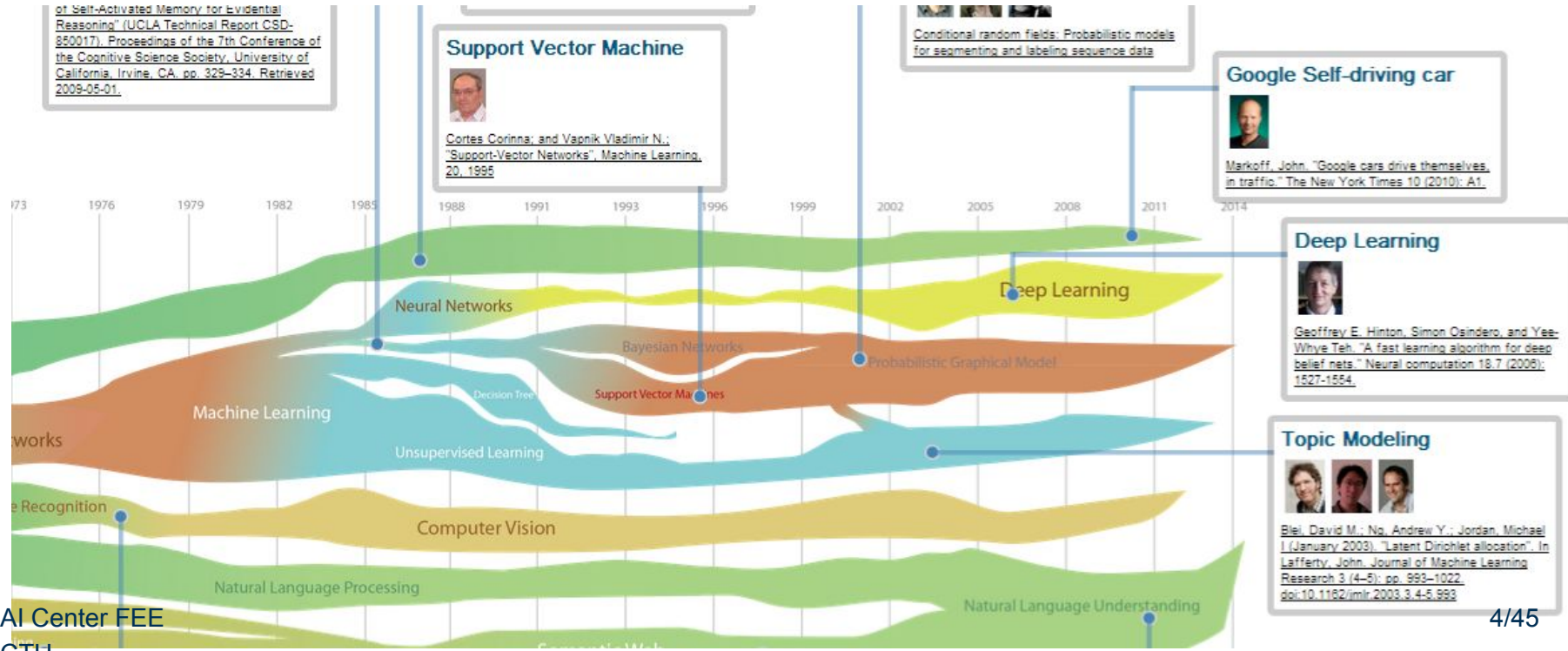


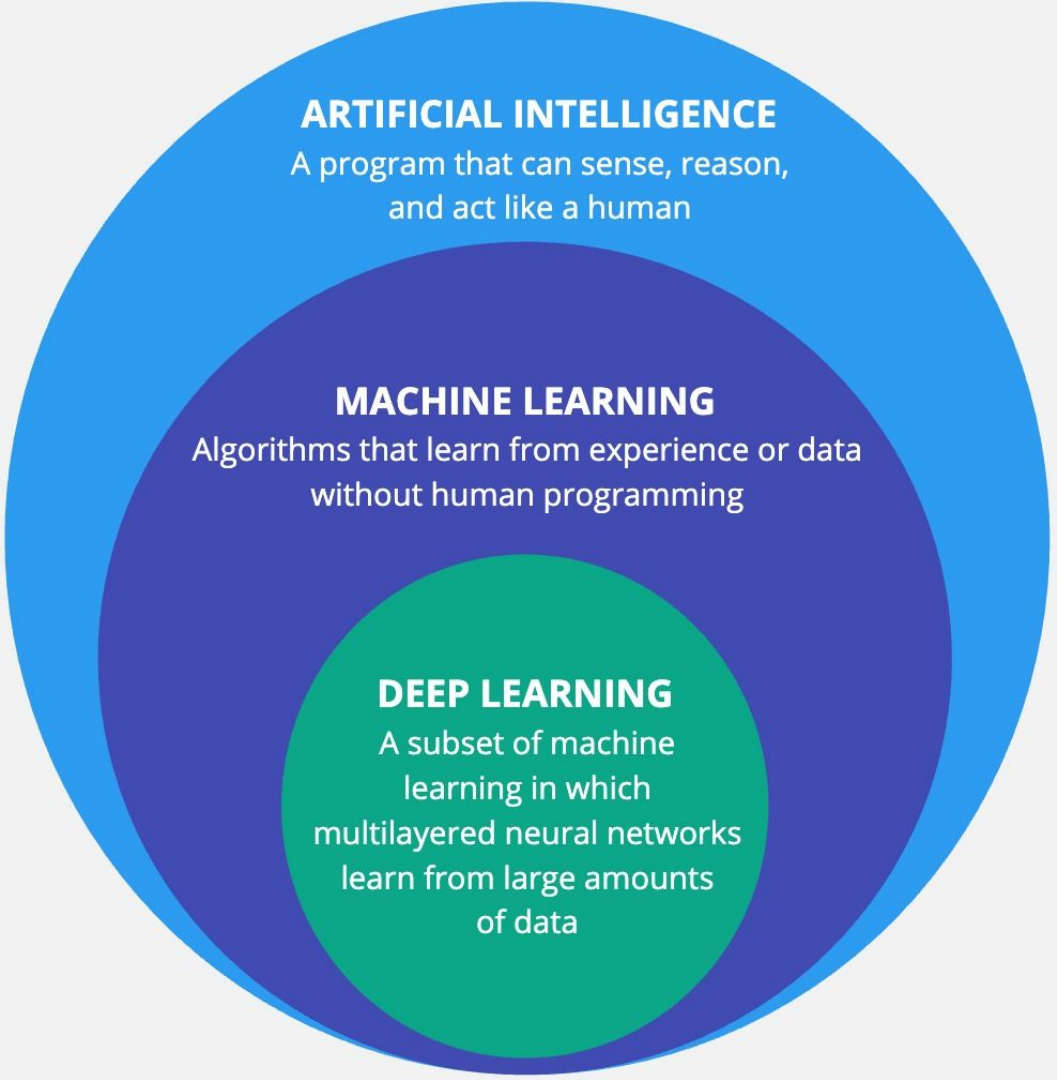
A Practical Guide to Machine Learning

Evolution of AI



Evolution of AI: Contd.





The diagram consists of three concentric circles. The outermost circle is light blue and contains the text 'ARTIFICIAL INTELLIGENCE' and its definition. The middle circle is dark blue and contains the text 'MACHINE LEARNING' and its definition. The innermost circle is green and contains the text 'DEEP LEARNING' and its definition. The circles are nested, indicating that Deep Learning is a subset of Machine Learning, which is a subset of Artificial Intelligence.

ARTIFICIAL INTELLIGENCE

A program that can sense, reason,
and act like a human

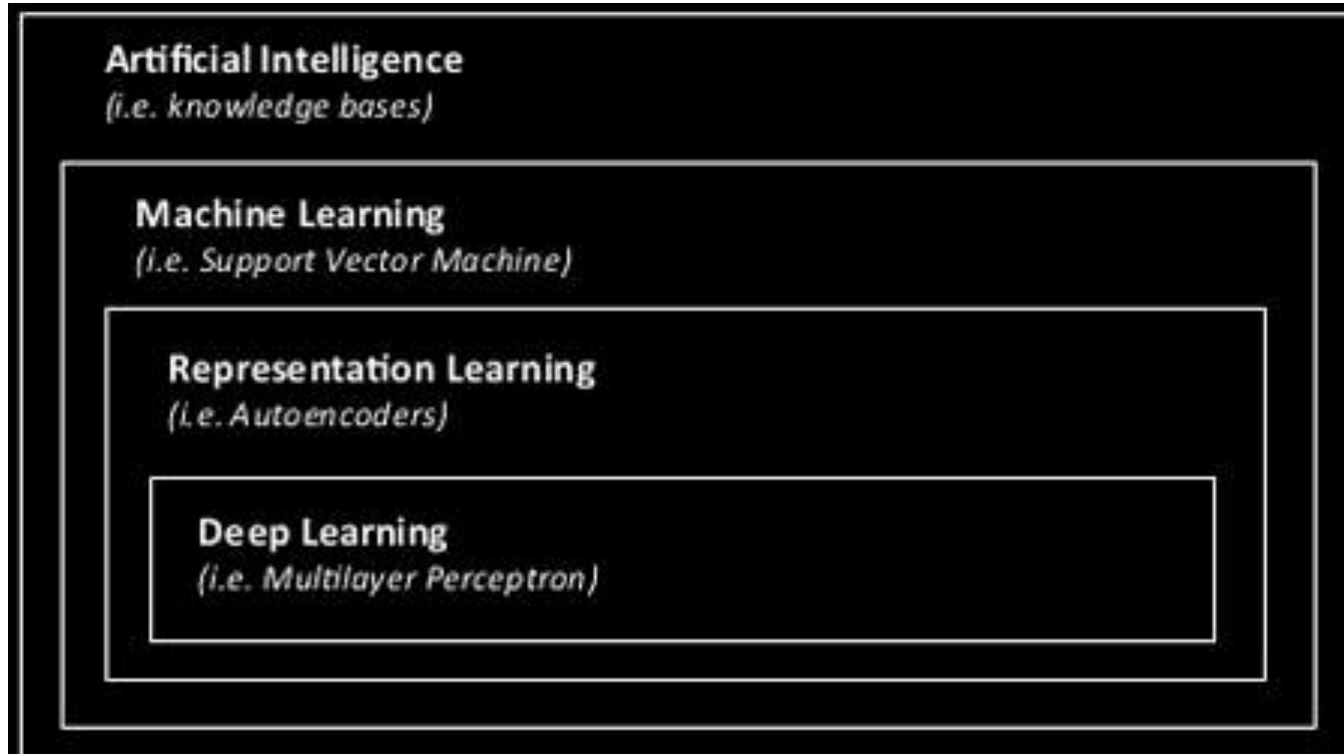
MACHINE LEARNING

Algorithms that learn from experience or data
without human programming

DEEP LEARNING

A subset of machine
learning in which
multilayered neural networks
learn from large amounts
of data

Relation between AI, ML and DL



Why Care AI now: Garter Hype Cycle

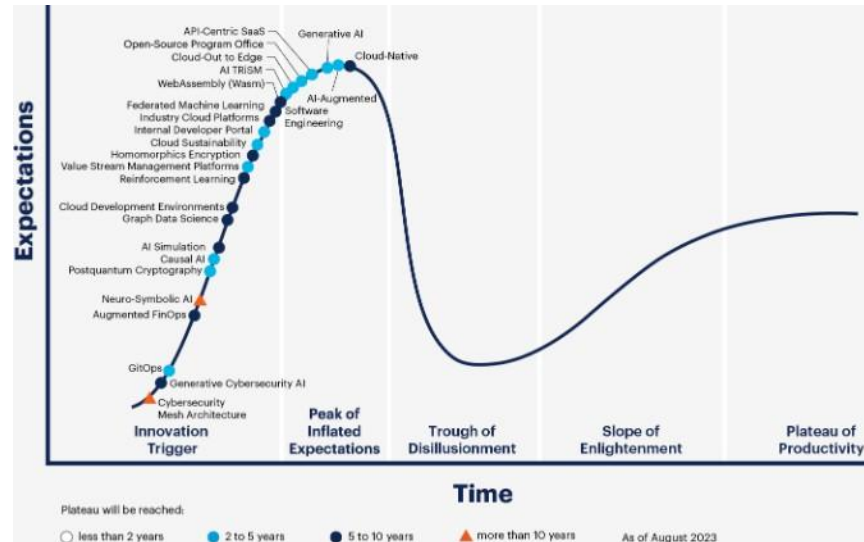


Figure: Hype Cycle for Emerging Technologies, 2023 Source:Garter.com

Emergent AI: Rank 1 in AI skills in Demand



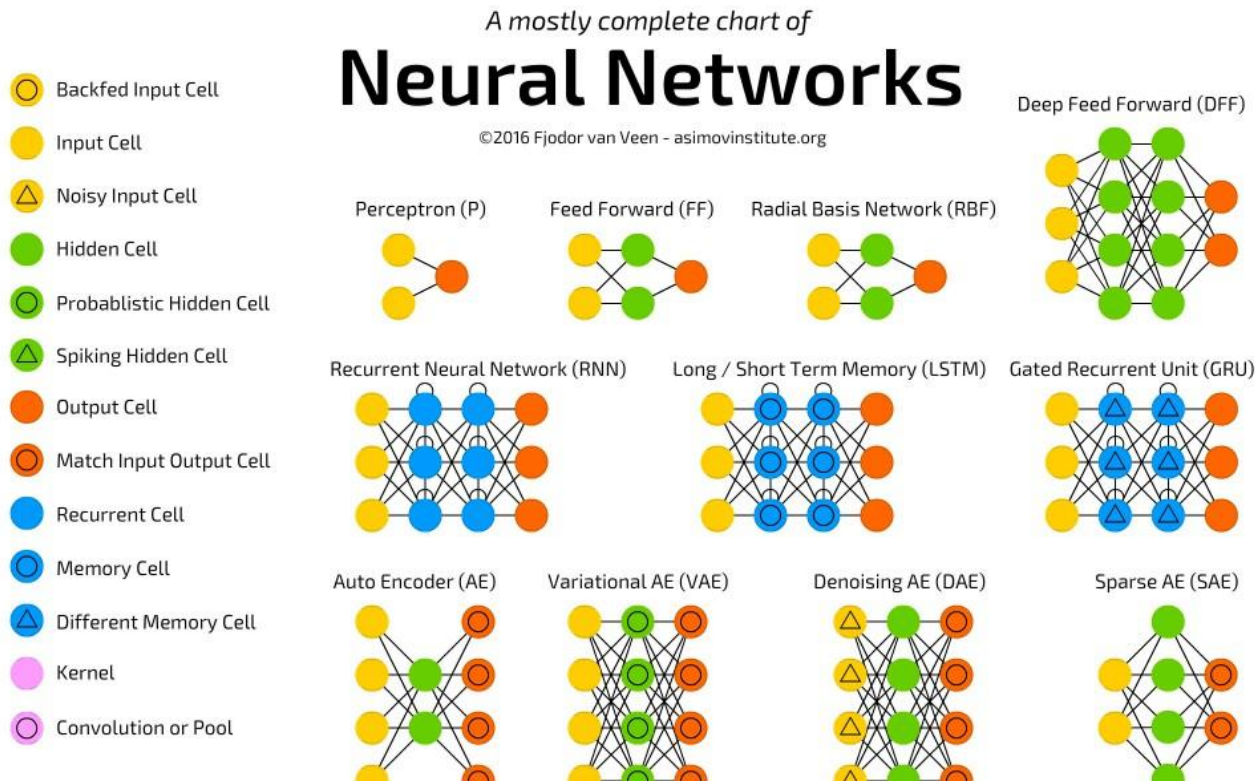
- Federated machine learning
- Graph data science (GDS)
- Reinforcement learning (RL)
- Generative AI
- Causal AI
- Explainable AI

Why these are Prominent (Need) Now

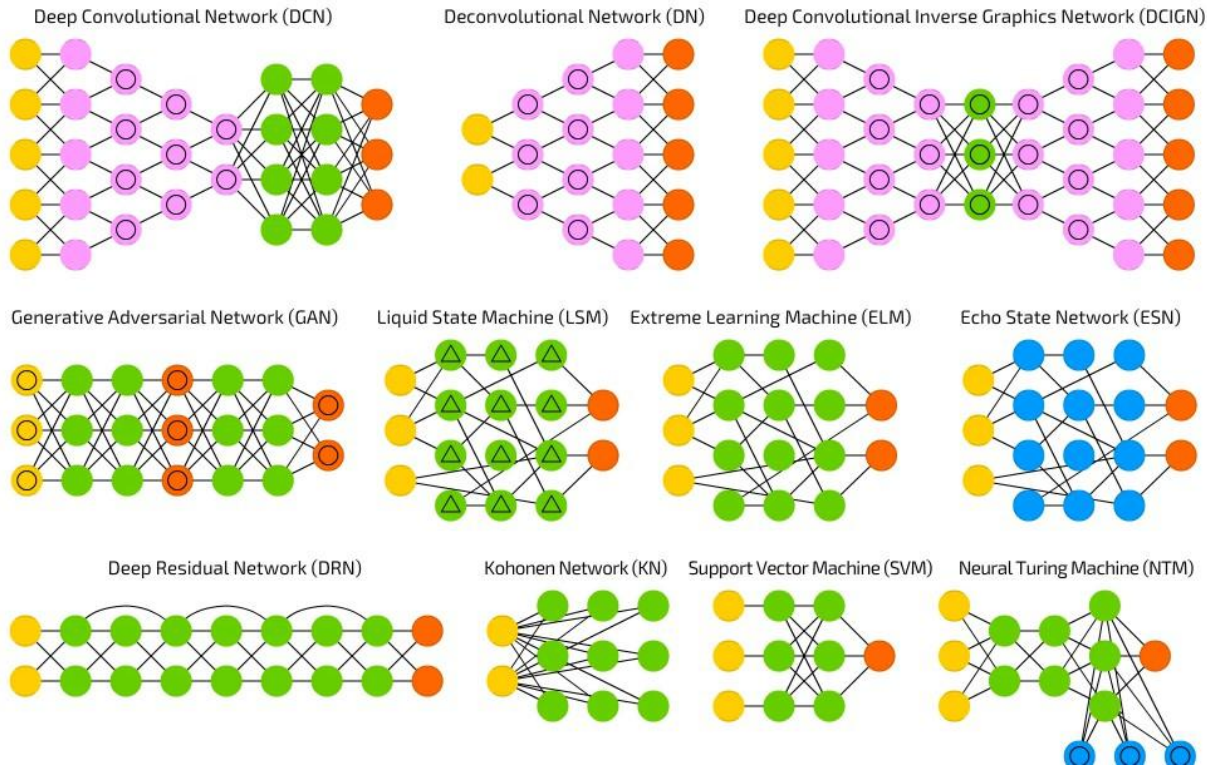


- Deep Learning
- Most mathematical foundations were laid before 2000
- Post 2000: Availability of Large data and computer power ($\square$ of cores ~~es~~[#] packed on single Processing Unit)
- Majorly driven by Neural Network
- Note: Neural Networks is not Limited to rosenbelt Neuron (Weighted summation with activation) and Multi layer perceptron
- Basic Neuron (Brick) can be rosenbelt Neuron, RNN, LSTM, CNN, etc
- Architectures (Wall) can be MLP, Auto encoder, Encoder decoder, RBM, SOM, etc

Neural Networks: Cheat sheet



Neural Networks: Cheat sheet Contd.





Emergence of Machine Learning to Deep Learning

Programming Vs ML

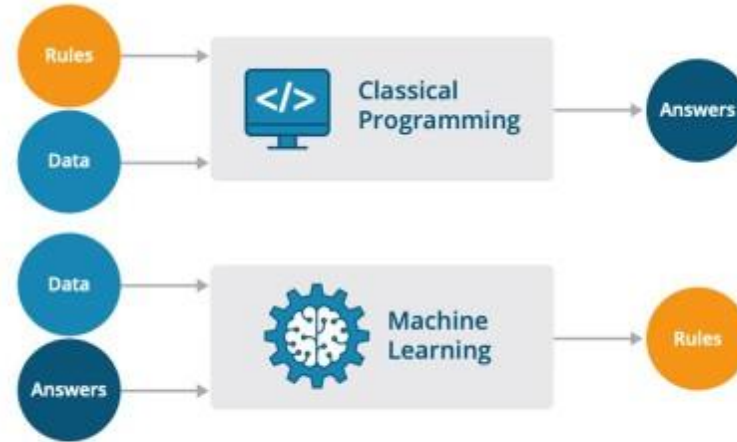


Figure: Basic view Given input Data (X) and output data (Y) then ML learn the rules. Note: Need and nature of Y is subjective to learning. Will be clear in subsequent slides

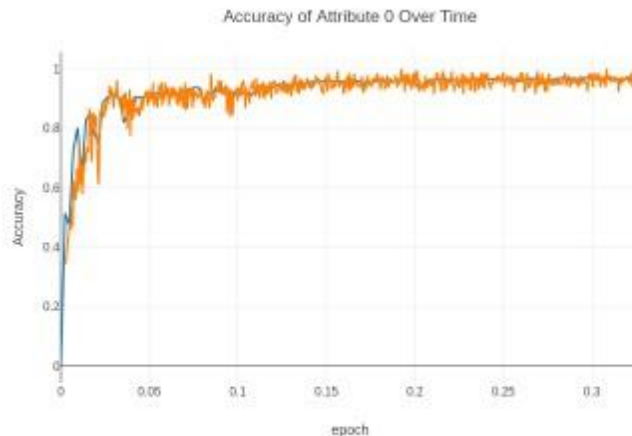
After Training; ML Models are used to predict answers



Machine Learning



- “Learning is any process by which a system improves performance from experience.” - Herbert Simon
- ”Machine Learning is the study of algorithms that improve their performance P at some task T with experience E ” - Tom Mitchel



Decimation of a Task into ML nomenclature



- T: Playing checkers
 - P: Percentage of games won against an arbitrary opponent
 - E: Playing practice games against itself
- T: Recognizing hand-written words
 - P: Percentage of words correctly classified
 - E: Database of human-labeled images of handwritten words
- T: Predicting an apple price
 - P: How close is predicted price close to actual price
 - E: Database of input features like size, area, colour, etc and corresponding prices for 'n' number of apples

Data



← Features →					Label
Position	Experience	Skill	Country	City	Salary (\$)
Developer	0	1	USA	New York	103100
Developer	1	1	USA	New York	104900
Developer	2	1	USA	New York	106800
Developer	3	1	USA	New York	108700
Developer	4	1	USA	New York	110400
Developer	5	1	USA	New York	112300
Developer	6	1	USA	New York	114200
Developer	7	1	USA	New York	116100
Developer	8	1	USA	New York	117800
Developer	9	1	USA	New York	119700
Developer	10	1	USA	New York	121600

Figure: Features are input (X) and Output/Label is Y. Out of 'n' samples ie no. of Rows 80% are for Training and 20 % are for validation

The Machine Learning Pipeline

1) Input Data:

Each data sample can be: $\{X\}$, $\{X, Y\}$, $\{X, Y, \text{Model}\}$)

2) Preprocessing :

Missing values, noise, and inconsistencies

Normalize/standardize features

Convert categorical variables Y (e.g., one-hot encoding)

Split into training/ validation/

3) Choose Model Architecture:

SVM, Neural Networks Decision Trees

A function $f()$ which relates

$X \rightarrow Y_{\text{hat}}$

Ensemble models if problem

4) Define Loss Function:

Disparity between Y & Y_{hat}

5) Train Model via Optimization:

Four Pillars of Machine Learning

Data: Processed data for training (each sample is $\{X, y\}$ & y is continuous)

Model (Hypothesis): Function mapping input to output

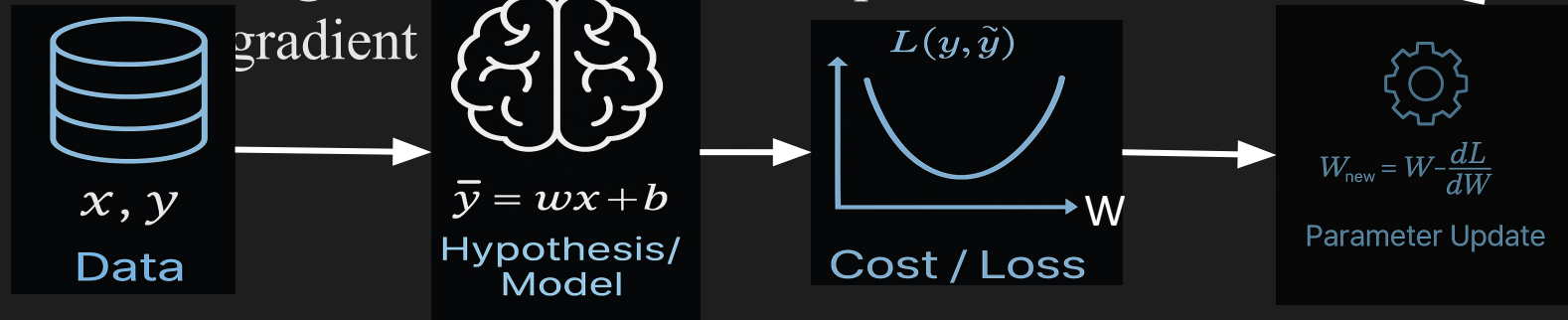
$$\hat{y} = f(X) = WX + w_0 = w_1x_1 + w_2x_2 + \dots + w_0$$

Loss Function: Measures model error

Regression loss function ex MSE

Learning Rule: Updates model parameters

$$w := w - \eta \cdot \frac{dL}{dw}$$



Data based ML categories

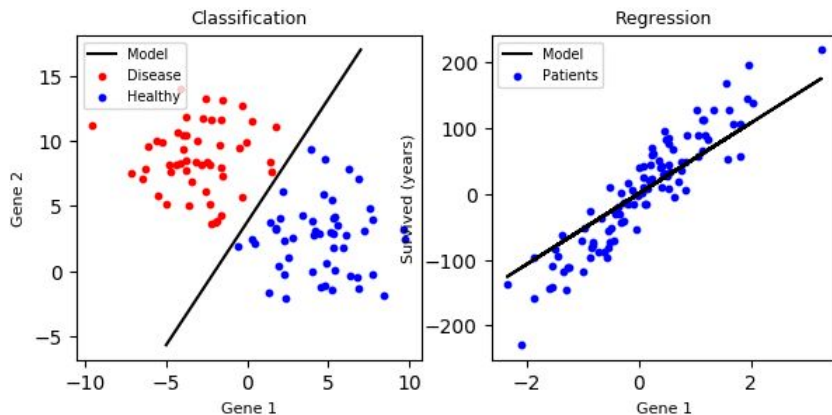


- Based on availability of Y OR Label
 1. Supervised Learning: each row has both feature vector and Label
 2. UnSupervised Learning: each row has only feature vectors
 3. Semi-Supervised Learning: Some rows have both feature vector& label.
Remaining rows have only feature vectors
 4. Reinforcement Learning: Environment is present; The agent/model interacts with environment and update weights based on feedback of interaction
 5. Self Supervised Learning: Leans underlying structures of data
 6. Active Learning: Some examples are picked form Humans to label
 7. Transfer Learning, Online Learning, Ensemble Learning Many more...
 8. The choice is subjective to kind of data acquired

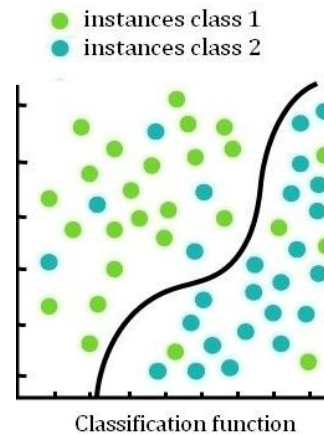
Data based ML categories



- Supervised Learning types: Based on Nature of Y
 1. Classification Task if Label OR Y is Discrete $Y=\{\text{Apple, orange}\}$ / Predicting apple or orange based on given fruit features.
 2. Regression Task if Label OR Y is continuous $Y=[a,b]$. . Predicting price range of an apple from given features.



(a) Regression vs Classification presuming a



(b) Classification presuming a non linear

2. Mathematical function OR Model



$X \rightarrow f() \rightarrow \hat{Y}$. Note X in each row of data is a 'd' dimensional feature vector and each vector is a concatenation of 'd' features $(x_1, x_2, \dots, x_d)$. $f()$ can be

- $WX + b$: Artificial neuron
- $W * X$: Convolutional neuron
- $(WX + b) < \text{thresh}$: 1950's Artificial neuron for classification
- $\tanh(Wx + b)$: Artificial neuron with non linear activation
- $aX^3 + bX^2 + c$: polynomial
- if else conditions: $x_1 < \text{thresh}_1, x_2 < \text{thresh}_2, x_3 < t3, \dots$ if Sunny is low, cloudy is high, wind is high then it will rain
- $p(x; \mu, \sigma)$: probalistic element
- RNN, LSTM: if input is atime series data

- And many more

Objective function



A measure of (dis)similarity between actual label (Y) and estimated label ($\hat{Y}$)

Loss function should be differentiable

- Metric need not be differentiable
- It depends on the TASK in hand
- This acts as Proxy of TASK to update weights so the MODEL performs better when deployed in real world

Loss Functions

- Regression Task
- Mean Square Error: $\frac{1}{n} \sum_{i=1}^n (\hat{Y}^i - Y^i)^2$
- Mean Absolute Error: $\frac{1}{n} \sum_{i=1}^n |\hat{Y}^i - Y^i|$
- SSIM: Variance along with Mean value
- KL Divergence: similarity between two distributions

Objective function

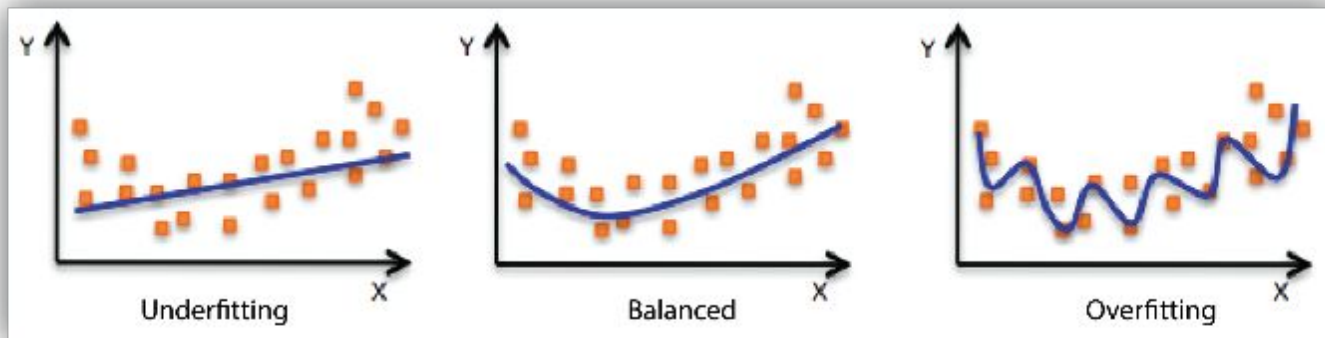


Loss Functions

- Classification Task

- Binary Cross Entropy (2class): $-(y \log(p) + (1 - y) \log(1 - p))$. p is scaled $\hat{y}$ to $[0,1]$
- hinge loss
- Multi class
- Gini impurity

Underfit Vs. Balancedfit Vs Overfit



Objective function

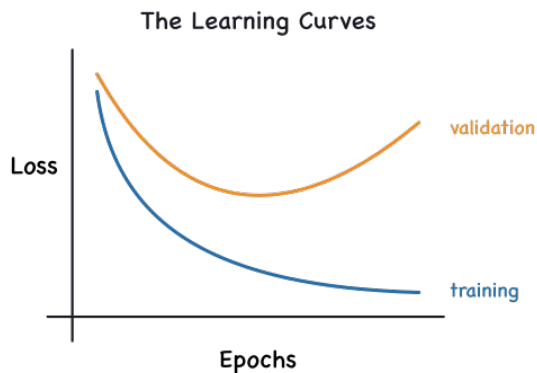


Figure: Identification of Underfit, Overfit, Balanced fit from Training Loss and Validation Loss curves

Increase the complexity of model to handle underfit

Regularizer to handle Overfit

- Assume $\hat{y} = WX$
- loss is MSE
- $J(W) = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2$
- $J(W) = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda ||W||_n$. L_n regularizer
- $J(W) = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda ||W||_2$. L₂ regularizer
- OR | Augment the data

4. Update of Learnable Parameters



Update Rule is subjective to model, Objective function and Task
McCulloch-Pitts Neuron for classification

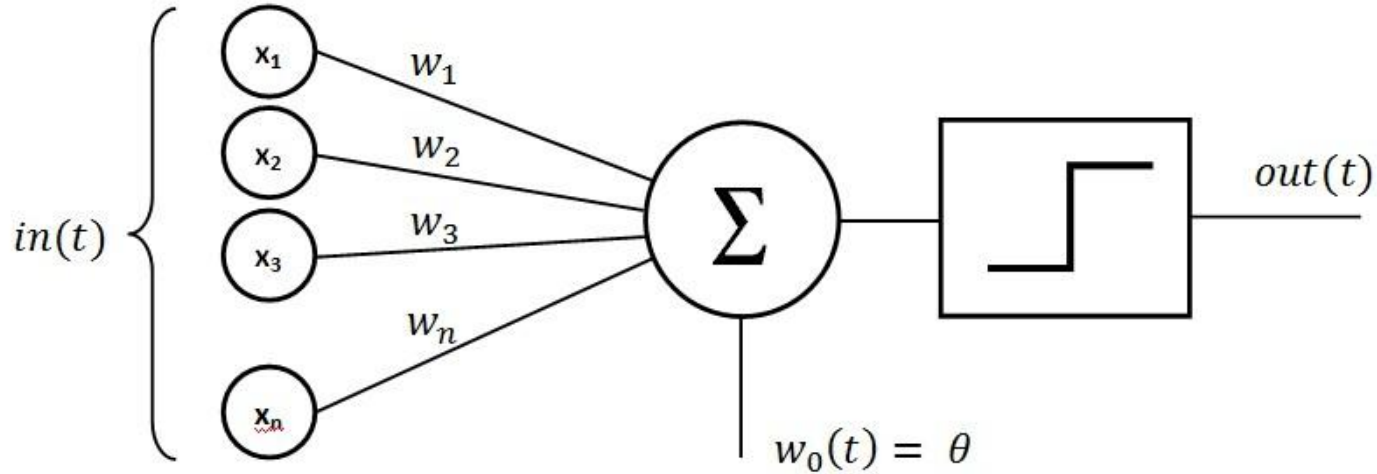
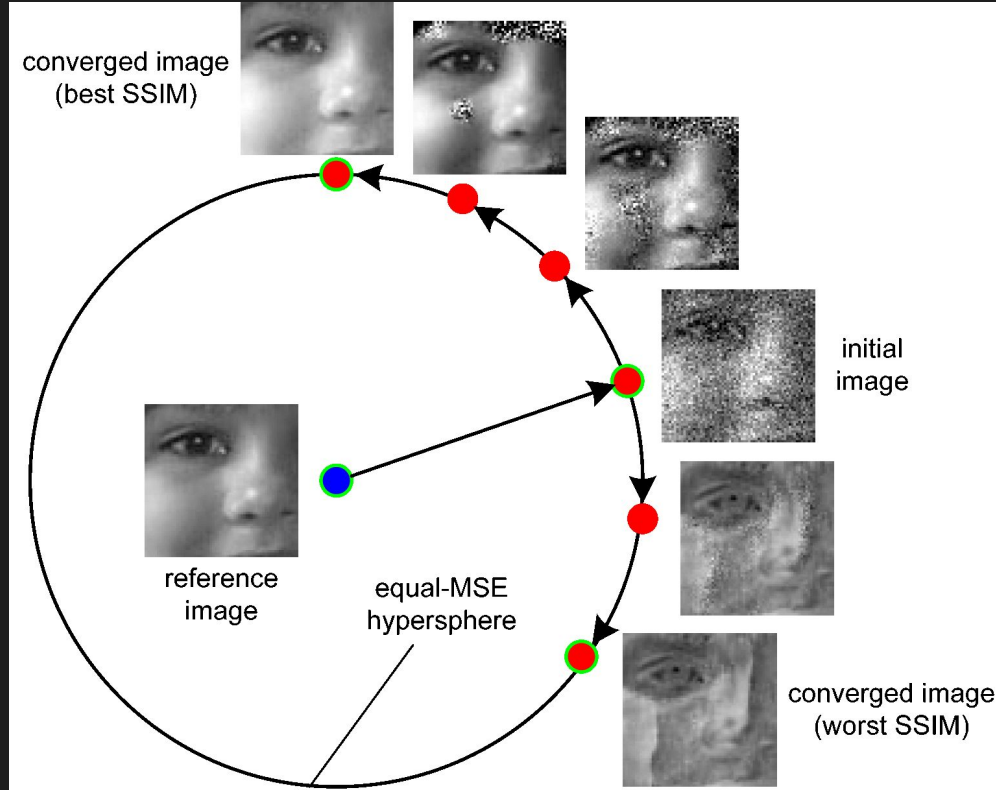
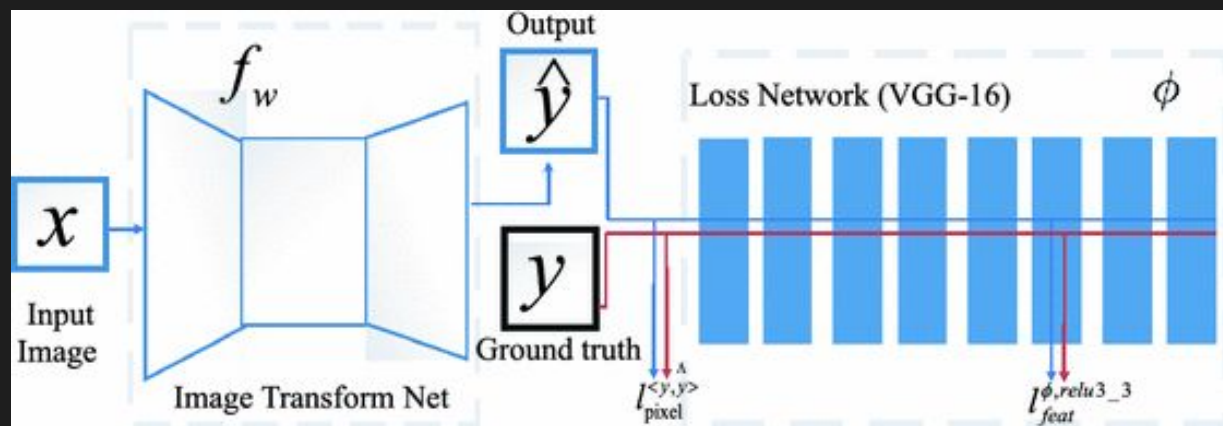
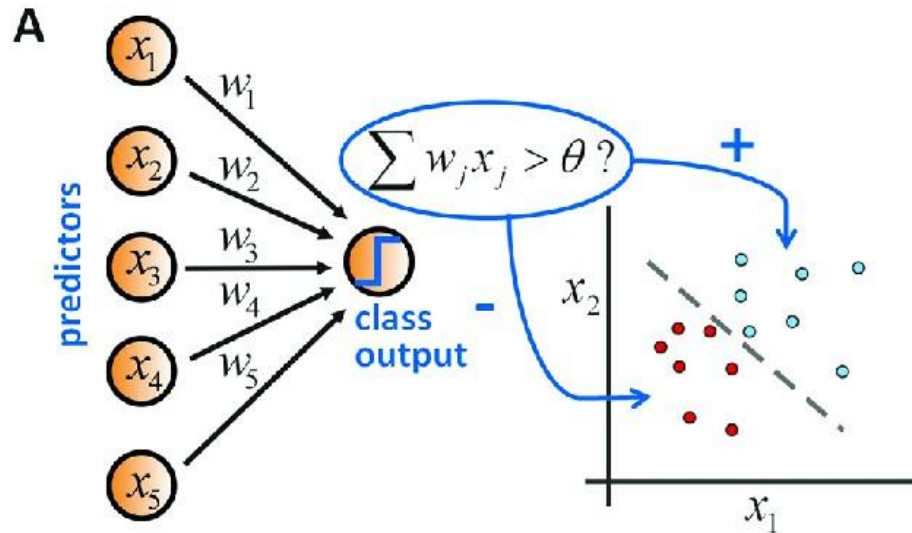


Figure: Weighted sum output is transformed using an activation function





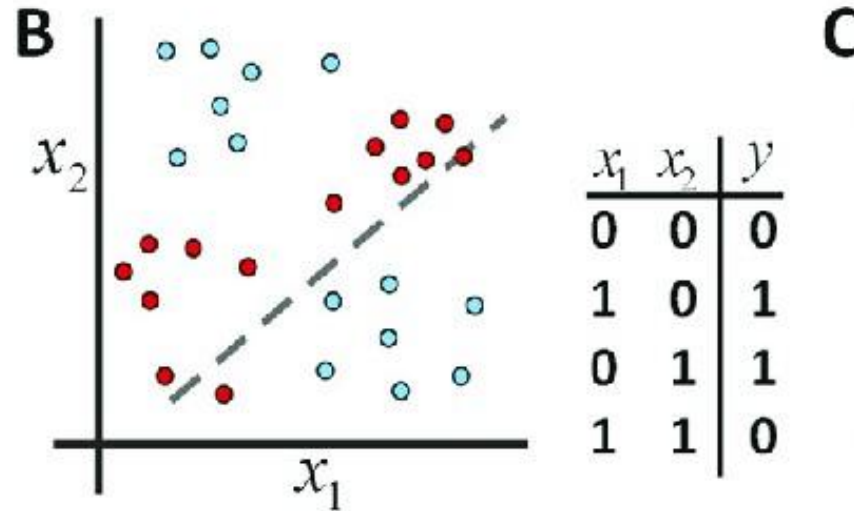
4. Update of Learnable Parameters



4. Update of Learnable Parameters



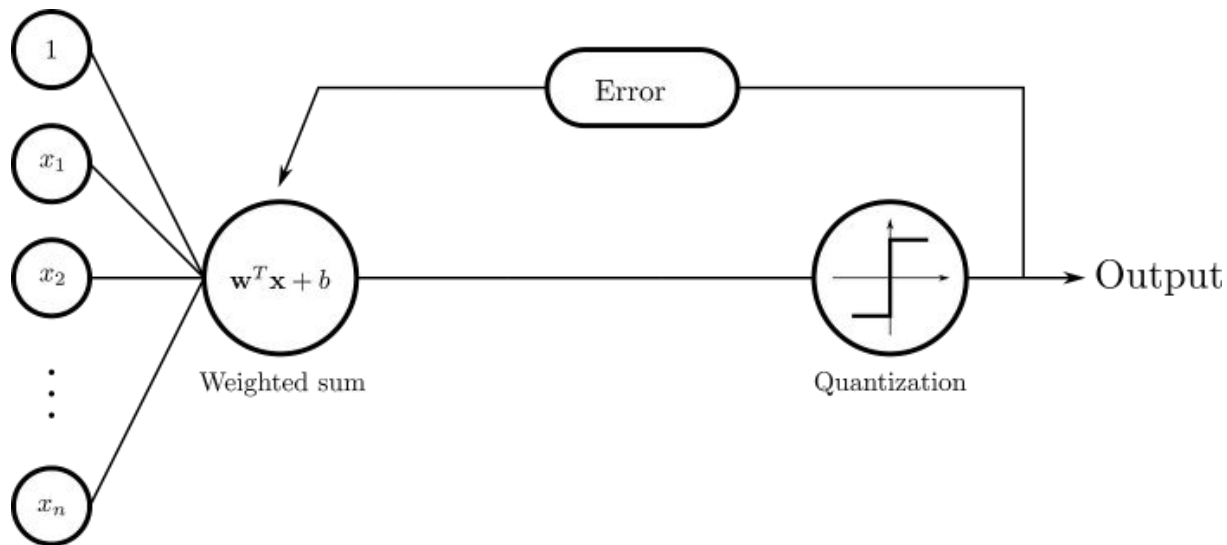
Weights are the tunable parameters which need to be learnt using training data. The initial values will be random



4. Update of Learnable Parameters



Update weights based on Error. Error is computed using Training data is only used to update Weights



4. Update of Learnable Parameters



Fundamental update rule of weights in Neural networks using Gradient descent

$$W_{\text{new}} = W_{\text{old}} - \alpha \nabla J(W_{\text{old}})$$

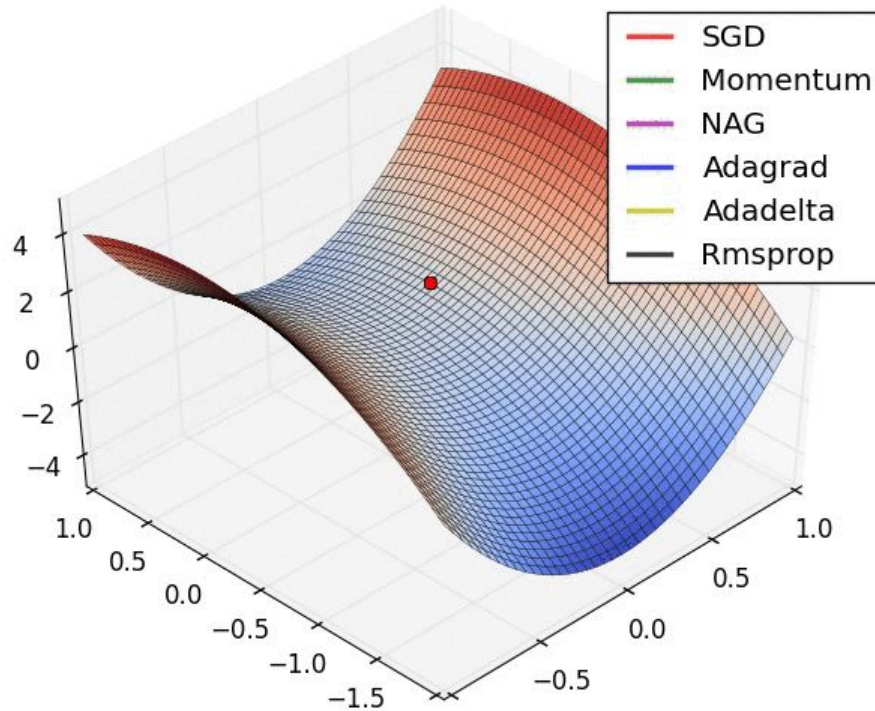
α : Step size

$$\nabla J(W_{\text{old}}) = \frac{\partial J(W_{\text{old}})}{\partial W_{\text{old}}}$$

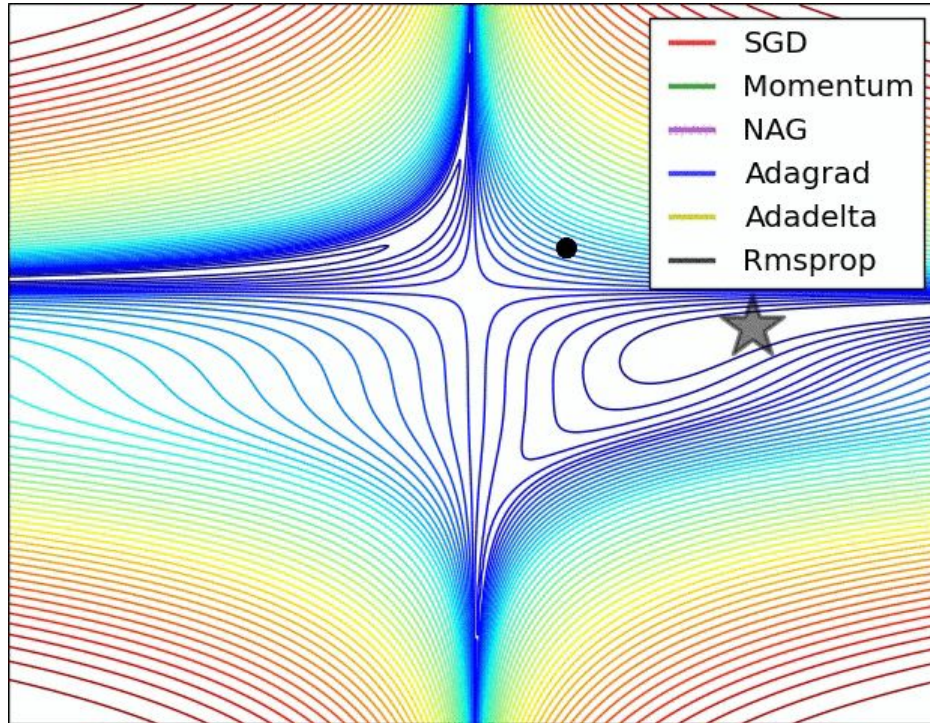
The update rule (presume stepsize=1) for above neuron and activation is

$$W_{\text{new}} = W_{\text{old}} \pm X \text{ When } \hat{Y} \neq Y \quad W_{\text{new}} = W_{\text{old}} + XY \text{ where } Y \in -1, 1$$

Choice of Optimizer



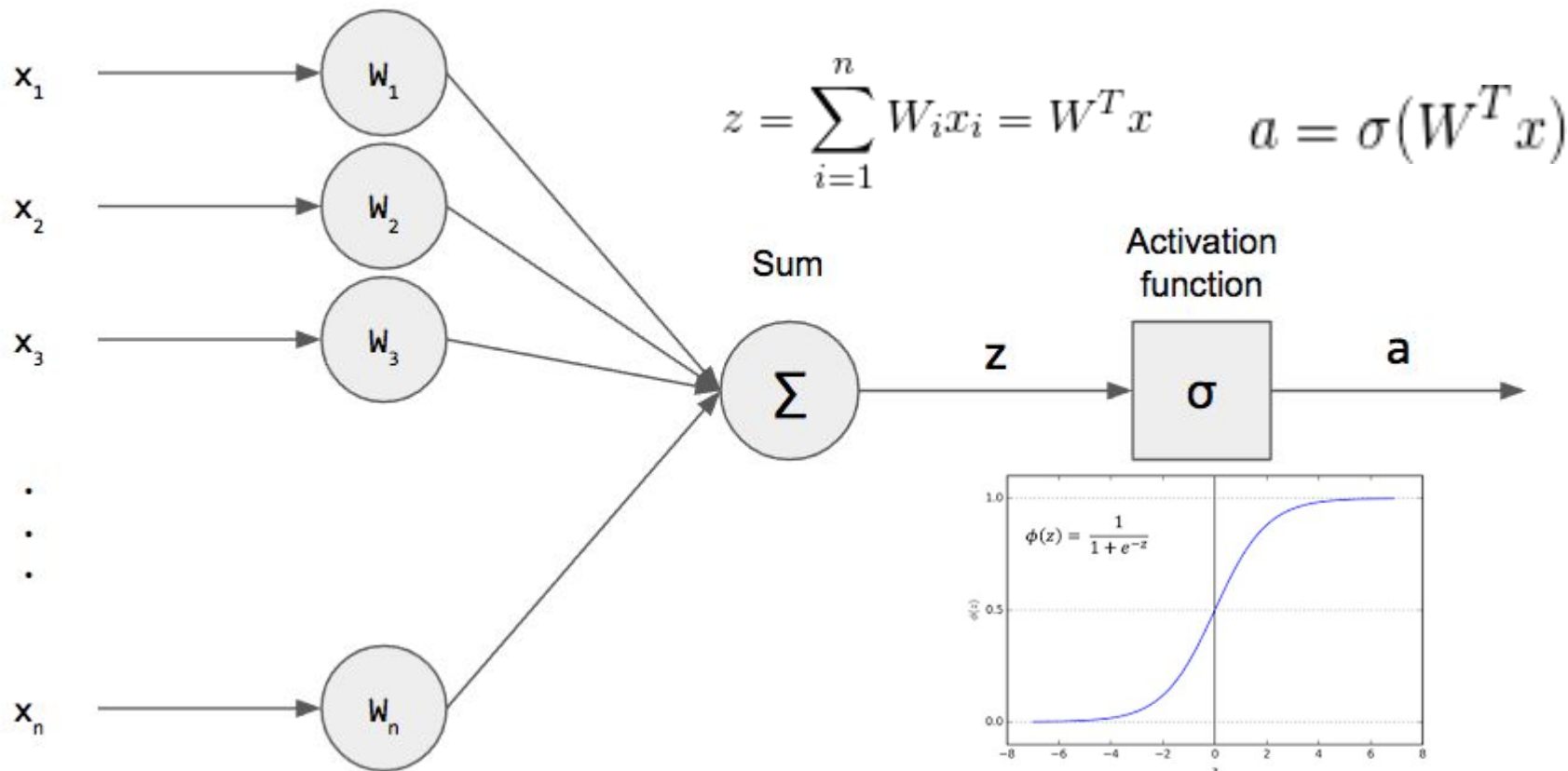
Choice of Optimizer

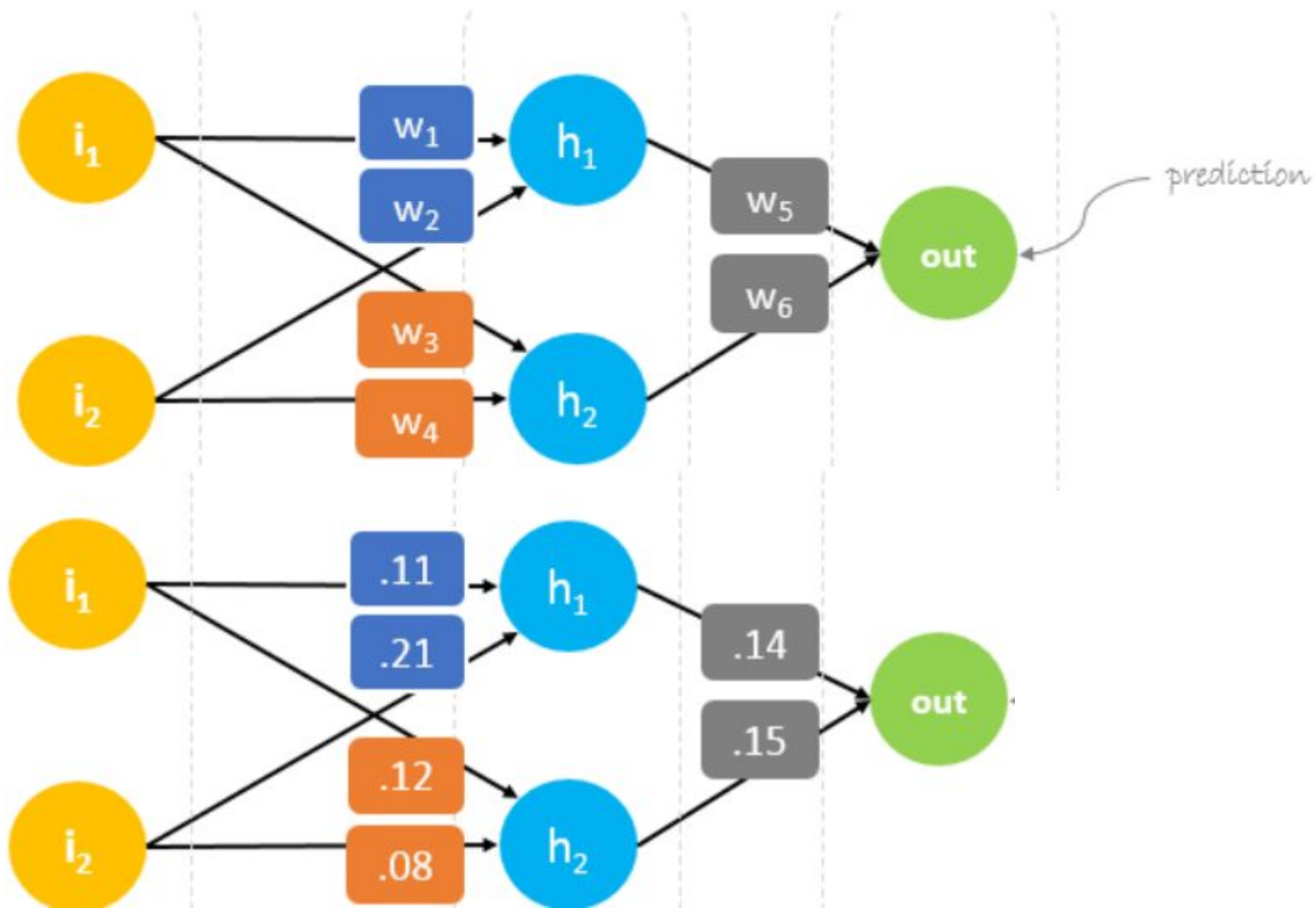


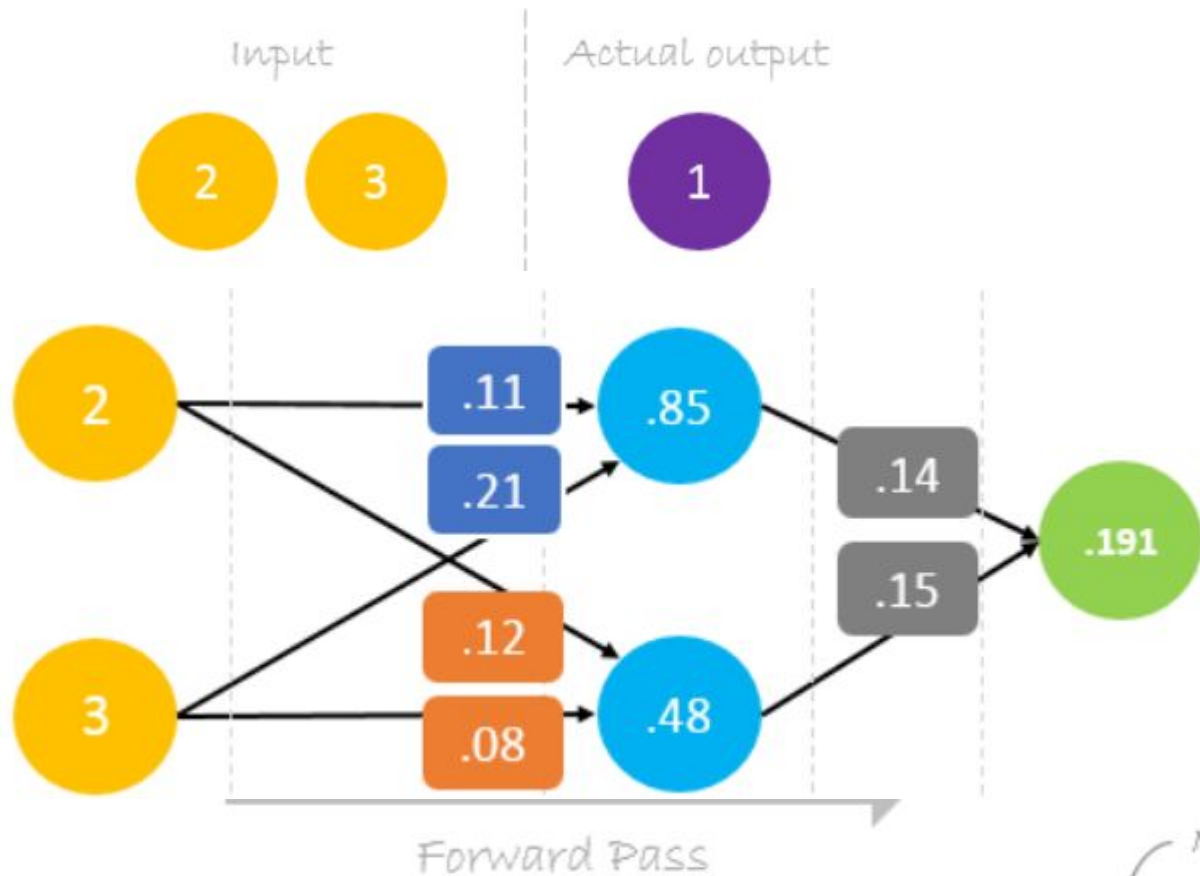
Algorithms

Inputs

Weights



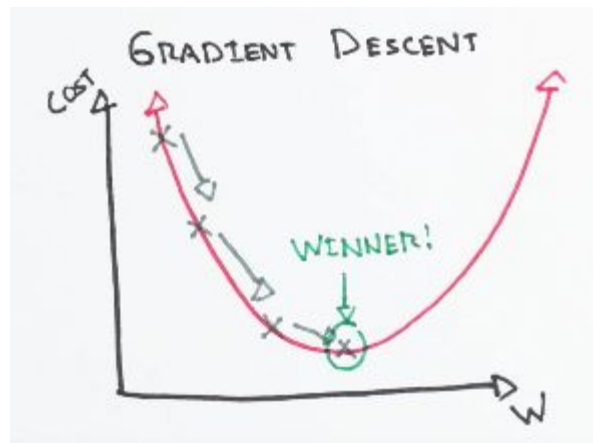




$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = \begin{bmatrix} 0.85 & 0.48 \end{bmatrix} \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.191 \end{bmatrix}$$

Matrix multiplication

Details



$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$= \frac{1}{2}(\mathbf{0.191} - \mathbf{1.0})^2 = \mathbf{0.327}$$

$$\text{prediction} = (\mathbf{h_1}) w_5 + (\mathbf{h_2}) w_6$$

$$\begin{aligned} h_1 &= i_1 w_1 + i_2 w_2 \\ h_2 &= i_1 w_3 + i_2 w_4 \end{aligned}$$

$$\text{prediction} = (\mathbf{i_1} \mathbf{w_1} + \mathbf{i_2} \mathbf{w_2}) w_5 + (\mathbf{i_1} \mathbf{w_3} + \mathbf{i_2} \mathbf{w_4}) w_6$$

$$^*W_x = W_x - \alpha \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

$$\frac{\partial \text{Error}}{\partial W_6} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial W_6}$$

$$\frac{\partial \text{Error}}{\partial W_6} = \frac{1}{2}(\text{predictoin} - \text{actula})^2 * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

$$\frac{\partial \text{Error}}{\partial W_6} = 2 * \frac{1}{2} (\text{predictoin} - \text{actula}) \frac{\partial (\text{predictoin} - \text{actula})}{\partial \text{predictoin}} * (i_1 w_3 + i_2 w_4)$$

$$\frac{\partial \text{Error}}{\partial W_6} = (\text{predictoin} - \text{actula}) * (h_2) \quad \frac{\partial \text{Error}}{\partial W_6} = \Delta h_2$$

$$*W_6 = W_6 - a \Delta h_2 \quad *W_5 = W_5 - a \Delta h_1$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\frac{\partial \text{Error}}{\partial W_1} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$

$$\frac{\partial \text{Error}}{\partial W_1} = \frac{\partial \frac{1}{2}(\text{predictoin} - \text{actula})^2}{\partial \text{prediciton}} * \frac{\partial (h_1) w_5 + (h_2) w_6}{\partial h_1} * \frac{\partial i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial \text{Error}}{\partial W_1} = 2 * \frac{1}{2}(\text{predictoin} - \text{actula}) \frac{\partial (\text{predictoin} - \text{actula})}{\partial \text{prediciton}} * (w_5) * (i_1)$$

$$\frac{\partial \text{Error}}{\partial W_1} = (\text{predictoin} - \text{actula}) * (w_5 i_1)$$

$$\frac{\partial \text{Error}}{\partial W_1} = \Delta w_5 i_1$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

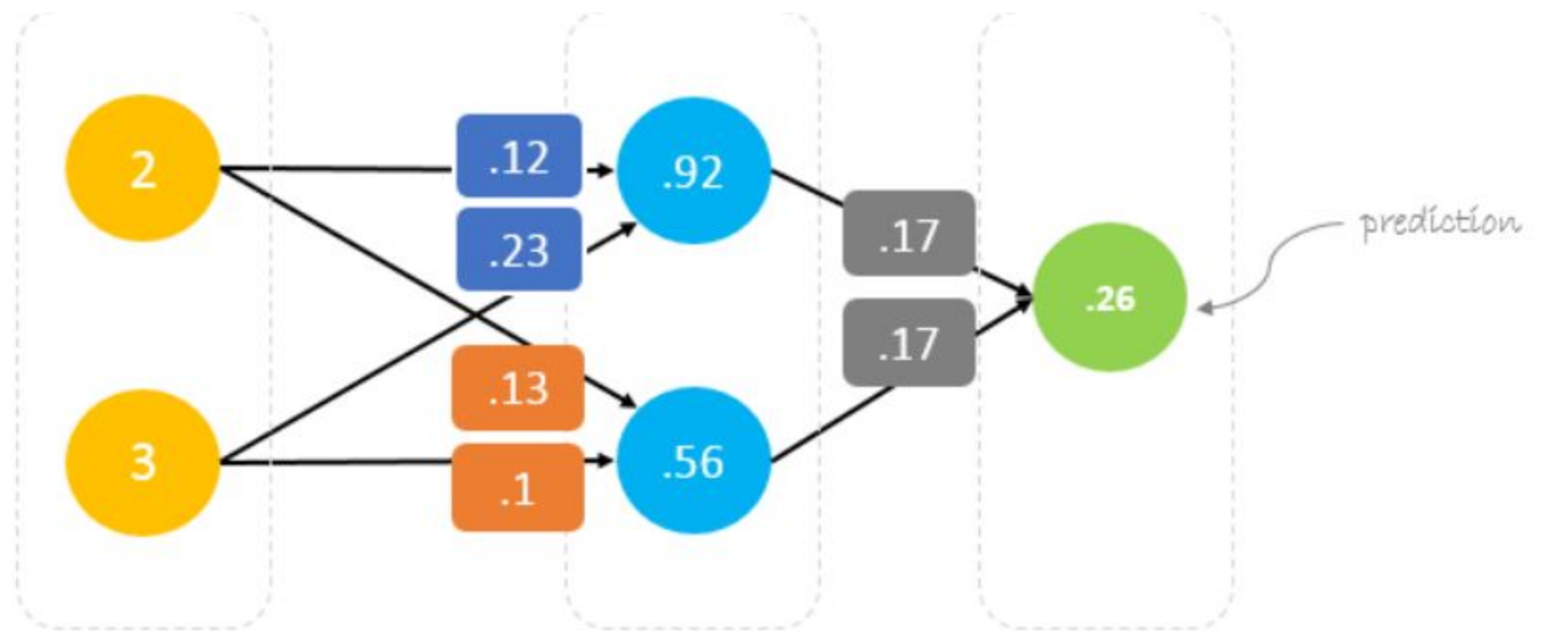
$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a \Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \cdot [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a i_1 \Delta w_5 & a i_1 \Delta w_6 \\ a i_2 \Delta w_5 & a i_2 \Delta w_6 \end{bmatrix}$$

$$\Delta = 0.191 - 1 = -0.809 \quad \leftarrow \text{Delta} = \text{prediction} - \text{actual}$$

$$a = 0.05 \quad \leftarrow \text{Learning rate, we smartly guess this number}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.011 \\ -0.017 & -0.017 \end{bmatrix}$$



Forward Pass

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.12 & 0.13 \\ 0.23 & 0.10 \end{bmatrix} = \begin{bmatrix} 0.92 & 0.56 \end{bmatrix} \cdot \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix} = \begin{bmatrix} 0.26 \end{bmatrix}$$

Matrix multiplication

Details

✓ Classification with perceptron

```
▶ X= np.array([1,2,3,4])  
y = np.array([-1, -1, 1,1])  
w = 0  
b = 0  
for iter in range(25):  
    y_hat = np.sign(w*X+b)  
    Correct = y==y_hat  
    for i in range(len(y)):  
        w = w +(y[i]-y_hat[i])*X[i]  
        b = b +(y[i]-y_hat[i])  
    print(Correct)  
print(w)  
print(b)
```

Classical ML Models

Linear Regression → Logistic Regression

$$f(x) = w_1x + w_0$$

$$f(x) = \sigma(w_1^T + b_1)$$

$$= \frac{1}{1 + e^{-(w_x + w_0)}}$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \lambda \underbrace{\sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

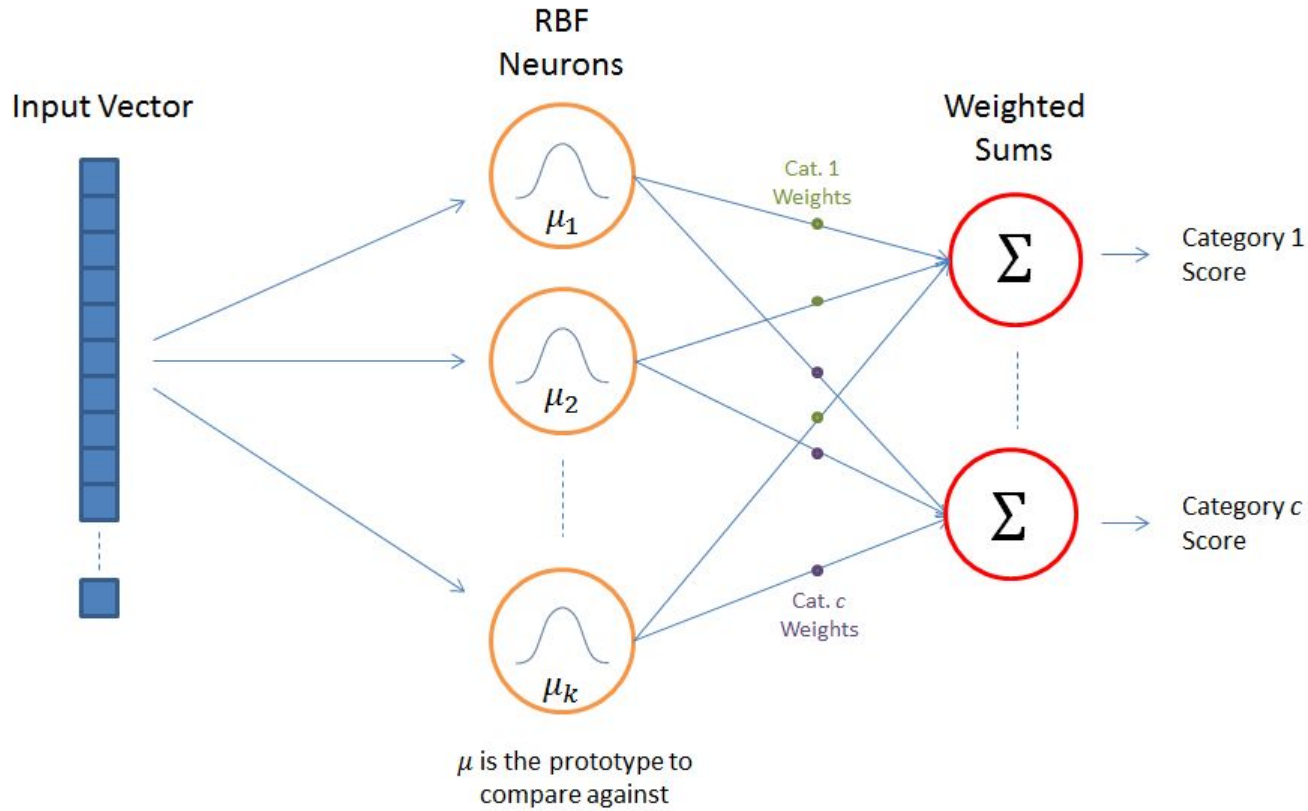
$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] + \lambda \|\mathbf{w}\|_2^2$$

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y^{(i)} f(\mathbf{x}^{(i)})) + \lambda \|\mathbf{w}\|_2^2$$

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i \left[y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) - 1 \right]$$

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \max(0, 1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b))$$

Classical ML Models: Kernel SVM mimicked with RBF neural network



Classical ML Models

Decision Trees → Softmax activation based neural networks

Entropy Based

$$H(p) = - \sum_{k=1}^K p_k \log(p_k)$$

$$IG(X) = H(\text{parent}) - \sum_i \frac{N_j}{N} H(\text{child}_j)$$

Gini Impurity

$$G(p) = 1 - \sum_{k=1}^K p_k^2$$

Neural Network

$$\mathcal{L}_{\text{CE}} = - \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)})$$

$$\mathcal{L}_{\text{Gini}} = \frac{1}{N} \sum_{i=1}^N \left(1 - \sum_{k=1}^K \hat{y}_k^{(i)^2} \right)$$

Classical ML Models

Eigen vectors → Autoencoders with linear activation

Clustering → Self organizing maps

Convolution → Convolutional neural networks

Dictionary learning → Encoder decoder architecture with regularizer

Probabilistic Graphical models → Graph neural networks

Feature extraction → Multi layer perceptron

Conditionanl random fields → Energy based neural networks (RNN using mean

Deep Neural Networks (DNNs)

Multiple layers = hierarchical representation or abstraction
Successive transformation of information
Enables learning of complex patterns

